

Deep Hedging in Low-Data Environments using Path Signatuaires

Nicholas Bossi

September 2024

Contents

List of Figures

1 Abstract

Historically, many approaches of hedging a financial derivative have been explored. This paper attempts to answer whether or not deep recurrent neural networks can provide robust tools for the hedging of a European Best-of derivative. As we are using deep learning techniques, we face the issue of not having sufficient real-world data in order to effectively train our model. We thus employ the use of a Conditional Variational Autoencoder as a tool to learn the underlying stock's distributions so that we can simulate a range of psuedo-real-world future paths. The advantage of this is two-fold, as it provides realistic training data for our deep hedger, as well as giving us a theoretical means to test the robustness of our final hedger under many future market conditions. Instead of directly using time-series stock data, we make use of a feature mapping for time-series data, known as a path signature, which has been shown to effectively capture key information of the joint-distributions of temporal data. Our findings imply that the use of deep recurrent networks give promising results in that they on average give a more risk-neutral hedging strategy than strategies informed by the Black-Scholes model, although issues in variability were also found.

2 Acknowledgements

I would like to thank Dr Jonathan Shock for agreeing to supervise me on this project, for asking thought-provoking questions, and for offering guidance on the process of scientific writing.

Special thanks must be given to Mr Yuri Robbertze, who went beyond the role of supervisor in helping me understand the topics involved in the project, and who's infectious zeal for all things maths continues to inspire me.

Finally, and as always, I thank my mom and dad for their continued emotional and financial support. Without them, I would never have been able to pursue my love for mathematics at a tertiary level, and for that I am eternally grateful.

3 Introduction

Fischer Black, Myron Scholes and Robert Merton came up a revolutionary model to price options in 1973 Black and Scholes 1973. They proved that under certain conditions, one can perfectly hedge an option by buying and selling the underlying asset. The model posits that the price of stocks follows a geometric brownian motion with constant drift and volatility. As with many models, key assumptions were made that fail in reality. Amongst others, these assumptions included the that the market is frictionless, and that assets can be traded continuously in time.

In reality assets cannot be traded continuously, and thus our trading strategy needs to somehow compensate for this fact, something which strategies informed by the Black-Scholes equations do not do.

This paper aims to assert whether deep neural networks can better manage portfolios containing the assets underlying a European call option so as to hedge said option. More explicitly, can a neural network govern a trading strategy which will more closely track the value of a contingent claim H than a trading strategy as motivated by the Black-Scholes equations. Due to the manner in which neural networks can infer complex relationships without explicit models of the underlying process, we hypothesise that by removing explicit model assumptions and the assumption of continuous trading, a neural network may be able to better hedge a financial instrument in the discrete-time case given that it does not have the restrictive assumptions of the Black-Scholes model.

We would like to employ deep learning techniques in order to hedge a financial instrument. A big hurdle to overcome is the fact that there is only one historical realisation of a particular financial instrument, and deep-learning techniques require large amounts of data to perform well. To overcome this we will use a generative model to learn the underlying distribution of the instrument of interest, and then generate many novel sample paths by sampling from this learnt distribution, thus enabling us to use more conventional deep learning techniques for the problem of hedging.

An added benefit of the generation result is that we can determine how well our model does on pseudo-real unseen data which can give us an indication on the robustness of our deep hedger.

4 Literature Review

4.1 Path Signatures

A path signature is an infinite dimensional tensor containing iterated path integrals over the domain of multidimensional paths (sequential data) Chevyrev and Kormilitzin 2016. It helps capture complicated interactions between paths, as well as fundamental characteristics of individual paths, such as when events took place relative to one another, whilst not being influenced by the parameterisation of the paths Lyons and McLeod 2023. It can be viewed as a feature map of a stream of data, and is essentially unique for a given path Lyons and McLeod 2023. H. Boedihardjo, Ni, and Qian 2013 Horatio Boedihardjo et al. 2014. Being a uniquely determined feature map, a natural question is to ask whether signatures can be used in a machine learning context, where feature extraction allows for more effective and efficient inference. Indeed they have been used in multiple machine learning problems with optimistic results. One successful application has been their use in classifying Chinese Characters, where the incorporation of terms from the path signature as features to a deep convolutional neural network (CNN) showed significant improvement in predictive capabilities. Graham 2013

Definition 4.1. A d -dimensional path $X : [a, b] \rightarrow \mathbb{R}^d$ is a continuous mapping from an interval of $\mathbb{R}$, $[a, b]$, to $\mathbb{R}^d$. Where X_t denotes $X(t)$ for $t \in [a, b]$.

Alternatively we can view a d -dimensional path as a collection:

$$X_t = (X_t^1, X_t^2, \dots, X_t^n),$$

where $X^i : [a, b] \rightarrow \mathbb{R}$ is a one-dimensional path for $\forall i \in \{1, \dots, d\}$.

Definition 4.2. Given a one-dimension path $X : [a, b] \rightarrow \mathbb{R}$, the *length* of X is denoted

$$|X| := \sup_{P \subset [a, b]} \sum_{i=1}^n |X_{t_i} - X_{t_{i-1}}| \quad (1)$$

where the supremum is taken over all finite partitions $P = \{x_1, \dots, x_n\}$ in $[a, b]$. A d -dimensional path $X = (X^1, X^2, \dots, X^n)$, is said to be of *bounded/finite variation* if $|X^i|$ is finite $\forall i \in \{1, \dots, d\}$.

4.1.1 Paths of finite variation

4.1.2 Path Integrals

Definition 4.3. Given a path of bounded variation $X : [a, b] \rightarrow \mathbb{R}$ and a function $f : \mathbb{R} \rightarrow \mathbb{R}$, the integral of f with respect to X over $[a, b]$ is defined by:

$$\int_a^b f(X_t) dX_t = \int_a^b f(X_t) \frac{dX_t}{dt} dt. \quad (2)$$

4.1.3 Path Signatures

Definition 4.4. Given a d -dimensional path of bounded variation, $X : [a, b] \rightarrow \mathbb{R}^d$, and recalling that for $\forall i \in \{1, \dots, d\}$, $X^i : [a, b] \rightarrow \mathbb{R}$ is a one-dimensional path. For all $i \in \{1, \dots, d\}$ we define the integral

$$S(X)_{a,t}^i = \int_{a < s < t} dX_s^i = X_t^i - X_a^i. \quad (3)$$

Similarly, for any pair $i, j \in \{1, \dots, d\}$ we define the double integral

$$S(X)_{a,t}^{i,j} = \int_{a < s < t} S(X)_{a,s}^i dX_s^j = \int_{a < s < t} \int_{a < r < s} dX_r^i dX_s^j = \int_{a < r < s < t} dX_r^i dX_s^j. \quad (4)$$

Continuing recursively, for any $k \geq 1$, and $i_1, i_2, \dots, i_k \in \{1, \dots, d\}$, we define the iterated integral

$$S(X)_{a,t}^{i_1, \dots, i_k} = \int_{a < s < t} S(X)_{a,t}^{i_1, \dots, i_{k-1}} dX_s^{i_k}. \quad (5)$$

The signature of a path $X : [a, b] \rightarrow \mathbb{R}^d$, denoted $S(X)_{a,b}$, is defined to be the infinite collection of all the iterated integrals of X . Formally, $S(X)_{a,b}$ is the sequence of real numbers

$$S(X)_{a,b} = \left(1, S(X)_{a,b}^1, \dots, S(X)_{a,b}^d, S(X)_{a,b}^{1,1}, S(X)_{a,b}^{1,2}, \dots \right) \quad (6)$$

where the superscripts iterate over all the multi-indexes

$$W = \{(i_1, \dots, i_k) | k \geq 1, i_1, \dots, i_k \in \{1, \dots, d\}\}. \quad (7)$$

4.1.4 Formal Power Series

Let $e_1, \dots, e_d$ be formal indeterminates. The algebra of formal power series in d indeterminates is the vector space of all series of the form

$$\sum_{k=0}^{\infty} \sum_{i_1, \dots, i_k \in \{1, \dots, d\}} \lambda_{i_1, \dots, i_k} e_{i_1} \dots e_{i_k}$$

where the second summation runs over all multi-indexes in W as above, and $\lambda_{i_1, \dots, i_k} \in \mathbb{R}$. A (non-commuting) formal polynomial is a formal power series for which only a finite number of coefficients $\lambda_{i_1, \dots, i_k}$ are non-zero. The terms $e_{i_1} \dots e_{i_k}$ are called monomials. The space of formal power series is frequently termed the tensor algebra of $\mathbb{R}^d$.

The tensor product $\otimes$ of monomials is defined as

$$e_{i_1} \dots e_{i_k} \otimes e_{j_1} \dots e_{j_l} = e_{i_1} \dots e_{i_k} e_{j_1} \dots e_{j_l}.$$

We can thus express the signature of a path X by a formal power series where the coefficient of each monomial $e_{i_1} \dots e_{i_k}$ is defined to be $S(X)_{a,b}^{i_1, \dots, i_k}$, and use the same symbol $S(X)_{a,b}$ to denote this representation

$$S(X)_{a,b} = \sum_{k=0}^{\infty} \sum_{i_1, \dots, i_k \in \{1, \dots, d\}} S(X)_{a,b}^{i_1, \dots, i_k} e_{i_1} \dots e_{i_k}.$$

4.1.5 Log-signatures

For a power series

$$x = \sum_{k=0}^{\infty} \sum_{i_1, \dots, i_k \in \{1, \dots, d\}} \lambda_{i_1, \dots, i_k} e_{i_1} \dots e_{i_k}$$

where $\lambda_0 > 0$, the logarithm of x is defined as

$$\log x = \log(\lambda_0) + \sum_{n \geq 1} \frac{-1^n}{n} \left(1 - \frac{x}{\lambda_0}\right)^{\otimes n}$$

where $\otimes n$ denotes the n -th power with respect to the product $\otimes$.

For a path $X : [a, b] \rightarrow \mathbb{R}^d$ we define the log signature of X to be the logarithm of the formal power series $S(X)_{a,b}$, $\log S(X)_{a,b}$. The log-signature offers an invertible transformation Zhou 2019 which drastically reduces the dimensionality of the feature space, and hence we will be using the log-signature rather than the signature as inputs to our market generator.

4.1.6 Lead-Lag Transformation

The notion of volatility is integral to the field of finance, and one widely used measurement of volatility is quadratic variation and cross-variation. We can use a lead-lag transformation of our data to ensure that we capture these variations Gyurkó et al. 2014. The lead-lag transformation is simple. Given a d dimensional stream of m -data points through time, $X = (x_1, \dots, x_m)$, we can define the lead transformation of component $j \in \{1, \dots, d\}$ to be

$$T_{lead}(X^j) = (x_1^j, x_2^j, x_2^j, x_3^j, \dots, x_m^j, x_m^j) = (s_1, \dots, s_{2m-1})$$

and the lag-transformation of component j to be

$$T_{lag}(X^j) = (x_1^j, x_1^j, x_2^j, \dots, x_{m-1}^j, x_{m-1}^j, x_m^j) = (t_1, \dots, t_{2m-1}).$$

The lead-lag transformation of component j is then defined to be the 2-dimensional stream, $T_{lead-lag}(X^j)$, whose k -th element is the tuple containing the k -th element of $T_{lead}(X^j)$ alongside the k -th element of $T_{lag}(X^j)$. That is

$$T_{lead-lag}(X^j) = ((s_1, t_1), \dots, (s_{2m-1}, t_{2m-1}))$$

4.1.7 Financial Market Simulation

Historically a multitude of methods have been employed to simulate financial markets. Classical approaches include those involving Geometric Brownian Motion and the Black-Scholes equation, Monte-Carlo Simulation, Heston models and autoregressive models. A drastic shift in classical methods of market simulation to the ones explored in this paper is that we no longer explicitly assume the underlying data generating distribution, but rather approximate this distribution implicitly, for example using a neural network. Buehler, Horvath, Lyons, Perez Arribas, et al. 2020

4.1.8 Financial Models/ Hedging Techniques

4.1.9 Machine Learning in small data environments

Typically the training of neural networks requires excessive amounts of data. Since we are limited by how long the price of a particular financial asset has existed/been recorder, there is often much less data available in finance then in other realms tackled by neural nets. In such low-data contexts, effective feature extraction is highly important in order to be able to condense the data available so that we can more easily learn useful functions relating our data to the output in question.

4.2 Stochastic Processes

Definition 4.5. Let $(\Omega, \mathcal{F}, \mathbb{F}, \mathbb{P})$ be a filtered probability space and $X = \{X_t : t \in \mathbb{I}\}$ a stochastic process. We term X an $(\mathbb{F}, \mathbb{P})$ martingale if

1. X is adapted to $\mathbb{F}$; i.e.: X_t is $\mathcal{F}_t$ measurable for $\forall t \in \mathbb{I}$,
2. $\mathbb{E}(|X_t|) < \infty$ for $\forall t \in \mathbb{I}$,
3. $\mathbb{E}(X_t | \mathcal{F}_s) = X_s \quad \mathbb{P} - a.s.$ for $s \leq t$

A stochastic process satisfying 1 and 2 and in addition satisfying

$$\mathbb{E}(X_t | \mathcal{F}_s) \geq X_s \quad (\text{resp. } \mathbb{E}(X_t | \mathcal{F}_s) \leq X_s) \quad \mathbb{P} - a.s. \quad \text{for } s \leq t$$

is known as a sub (resp. super) martingale.

Definition 4.6. Let $(\Omega, \mathcal{F}, \mathbb{F}, \mathbb{P})$ be a filtered probability space, $X = \{X_t : t \in \mathbb{N}\}$ a stochastic process and $\varphi = \{\varphi_t : t \in \mathbb{N}\}$ a predictable process (i.e.: φ_t is $\mathcal{F}_{t-1}$ for $\forall t$). Then the *martingale transform* of X by φ is the stochastic process $(\varphi \bullet X) = \{(\varphi \bullet X)_t : t \in \mathbb{N}\}$ where

$$\begin{aligned} (\varphi \bullet X)_0 &:= 0 \\ (\varphi \bullet X)_t &= \sum_{k=1}^t \varphi_k (X_k - X_{k-1}), t \geq 1. \end{aligned}$$

Theorem 1. Let $(\Omega, \mathcal{F}, \mathbb{F}, \mathbb{P})$ be a filtered probability space, $X = \{X_t : t \in \mathbb{N}\}$ a stochastic process and $\varphi = \{\varphi_t : t \in \mathbb{N}\}$ a bounded predictable process, then if X is a martingale, $(\varphi \bullet X)$ is also a martingale.

4.3 Trading

We will be focusing on trading in discrete time. In this context, let $T \in \mathbb{N}$ be the point in time up to which we consider trades/values of our portfolio. Assume we are working with $d + 1$ assets. The prices of these $d + 1$ assets are given by the stochastic processes $(B, S) = (B, S^1, \dots, S^d)$, where B represents the value of our bank account (also known as the *riskless* asset), and S^i represents the value of some financial asset.

Definition 4.7. Let $(\Omega, \mathcal{F}, \mathbb{F}, \mathbb{P})$ be a filtered probability space and assume that (B, S) is adapted to the filtration $\mathbb{F}$, then we term the tuple $((\Omega, \mathcal{F}, \mathbb{F}, \mathbb{P}), (B, S))$ a *market*.

Definition 4.8. Let N be an asset in our market. We can define all prices of our market (B, S) as the discounted prices $(Y, X) = (\frac{B}{N}, \frac{S}{N})$ where $X = (X^1, \dots, X^d)$ is defined such that $X^i = \frac{S^i}{N}$. When such an asset is used for discounting it is termed a *numeraire*. We will see how the use of numeraires can simplify calculations significantly.

Often B is chosen as the numeraire, resulting in our discounted prices being $(1, X)$. The measure under this numeraire is known as the *risk-neutral measure*, denoted P^* , whilst the measure under no numeraire is known as the *real-world measure*, denoted P .

Definition 4.9. Let φ_i^t be the number of units of asset i held between time $t - 1$ to time t and ν^t the number of units of the riskless asset held over the same period. Clearly we need to be able to know each of these holdings at time $t - 1$, thus we define a *trading strategy* to be the predictable process $(\nu, \varphi) = (\nu, \varphi^1, \dots, \varphi^d)$.

The *value* of our portfolio under the trading strategy (ν, φ) is thus the stochastic process $V((\nu, \varphi)) = \{V_t(\nu, \varphi) : t = 0, 1, \dots, T\}$ defined by

$$V_0((\nu, \varphi)) = (\nu_1, \varphi_1) \cdot (Y_0, X_0) = \nu_1 Y_0 + \sum_{i=1}^d \varphi_1^i X_0^i$$

and

$$V_t((\nu, \varphi)) = (\nu_t, \varphi_t) \cdot (Y_t, X_t) = \nu_t Y_t + \sum_{i=1}^d \varphi_t^i X_t^i \quad t \geq 1.$$

The *gains* associated with a strategy (ν, φ) is the stochastic process $G(\varphi) = \{G_t(\varphi) : t = 0, 1, \dots, T\}$ where

$$G_0(\varphi) := 0$$

$$G_t(\varphi) = \sum_{k=1}^t \varphi_k \cdot (X_k - X_{k-1}) \quad t \geq 1$$

Note that gains are only associated with changes in the non-riskless assets $X^1, \dots, X^d$, thus we see that by knowing the initial value of the portfolio and $\varphi^1, \dots, \varphi^d$ we can in fact recover the amount that must be invested in the riskless account via the formula

$$V_0((\nu, \varphi)) + G_t(\varphi) = \nu_t + \sum_{i=1}^d \varphi_t^i X_t^i.$$

Thus, specifying the initial value v_0 and the predictable holdings of the non-riskless assets $\varphi = (\varphi^1, \dots, \varphi^d)$ uniquely specifies the full strategy. Consequently, we subsequently denote strategies by (v_0, φ) .

Definition 4.10. A trading strategy φ is termed an arbitrage strategy if

$$V_0(\varphi) = 0, V_T(\varphi) \geq 0 \quad \mathbb{P}\text{-a.s. and } \mathbb{P}(V_T(\varphi) > 0) > 0.$$

A market model is called arbitrage free if there does not exist any arbitrage strategies.

Definition 4.11. Let $(\Omega, \mathcal{F}, \mathbb{P})$ be a measure space and X a random variable. A probability measure $\mathbb{P}^*$ on $(\Omega, \mathcal{F})$ is termed an *Equivalent Martingale Measure* (EMM) for X if

1. $\mathbb{P}^*$ is equivalent to $\mathbb{P}$, i.e.:

$$\mathbb{P}^*(A) = 0 \Leftrightarrow \mathbb{P}(A) = 0 \quad \text{for } \forall A \in \mathcal{F}$$

2. X is an $(\mathbb{F}, \mathbb{P}^*)$ -martingale.

We denote $\mathcal{P}$ to be the set of EMMs for X , and $\mathbb{E}^*$ to be the expectation w.r.t $\mathbb{P}^*$

Now let us consider our portfolio of assets X and let $\mathbb{P}^*$ be an EMM for X . Let φ be our strategy and assume $V_0(\varphi) = 0$. By ?? we have that $V_t(\varphi) = V_0(\varphi) + \sum_{k=1}^t \varphi_k (X_k - X_{k-1})$ is a martingale transform and so $V(\varphi)$ is a martingale under $\mathbb{P}^*$. Consequently we have that

Theorem 2. (FTAP1). For a financial market $(\Omega, \mathcal{F}, \mathbb{F}, \mathbb{P}), X$, the following are equivalent (t.f.a.e):

1. There are no arbitrage opportunities;
2. $\mathcal{P} \neq \emptyset$.

4.4 Financial Options

Definition 4.12. A *contingent claim* H is an $\mathcal{F}_T$ -measurable random variable. We term a contingent claim H a *derivative* of some underlying asset X if H is measurable with respect to $\mathcal{F}_T^X$.

Issuers of contingent claims view them as liabilities that expire at time T . Thus in order to safeguard against the stochastic outcomes of H the issuer would like to *replicate* the price of the contingent claim by trading the assets underlying the claim.

Definition 4.13. A strategy φ s.t $V_T(\varphi) = H$ $\mathbb{P}$ -a.s. is termed a *replicating strategy/portfolio*. If such a strategy exists for a given contingent claim H , then we term H *attainable*. A market is said to be *complete* if every contingent claim is attainable.

Theorem 3. (FTAP 2). Given a financial market $(\Omega, \mathcal{F}, \mathbb{F}, \mathbb{P}), X$ with no arbitrage opportunities, t.f.a.e:

1. The market is complete
2. $|\mathcal{P}| = 1$, i.e. there is a unique EMM.
3. X has the predictable representation property (PRP) with respect to every $P^* \in \mathcal{P}$. That is, every $(\mathbb{F}, \mathbb{P}^*$ -martingale M with $M_0 = 0$ can be written as a martingale transform w.r.t X .

Definition 4.14. European Call Option.

Let S be a stock and S_t the value of the some stock at time t . A *European call option* is a derivative of S of the form

$$H = (S_T - K)^+$$

where K is known as the strike price of the option. What this option represents is contract whereby the purchaser of the option has the right to buy the underlying stock for price K at time T , generally done when $S_T \geq K$, and consequently meaning that the effective value of the option is given by H . European options differ from American options in that they can only be exercised at maturity (time T).

Definition 4.15. A *best of European option* is an option of the form

$$H = \max\{H_T^1, H_T^2\}$$

where H^1 and H^2 are functions of two stocks S^1 and S^2 .

For example:

$$H = \max\left\{\frac{S_T^1}{S_0^1}, \frac{S_T^2}{S_0^2}\right\}$$

is the option whose payoff corresponds to the maximum of the relative increases in prices of stocks 1 and 2.

4.5 Stochastic Calculus

Definition 4.16. Let $(\Omega, \mathcal{F}, \mathbb{F}, \mathbb{P})$ be a filtered probability space. A stochastic process $W = \{W_t : t \geq 0\}$, is called a *Brownian motion* (BM) w.r.t $\mathbb{F}$ if it satisfies the following properties:

1. $W_0 = 0$ $\mathbb{P}$ a.s.
2. W_t has independent increments; i.e., for every $0 \leq t_1 < t_2 < \dots < t_n$, the random variables

$$W_{t_1} - W_{t_0}, W_{t_2} - W_{t_1}, W_{t_n} - W_{t_{n-1}}$$

are independent.

3. W_t has stationary, normally distributed increments, namely:

$$W_t - W_s \sim \mathcal{N}(0, t - s)$$

for $s < t$

4. $W_t - W_s$ is independent of $\mathcal{F}_s$ for $s < t$.

Proposition 1. Let W^i and W^j be two BM's on some filtered probability space and $s < t$, then

$$Cov(W_t^i - W_s^i, W_t^j - W_s^j) = \rho \sqrt{Var(W_t^i - W_s^i)} \sqrt{Var(W_t^j - W_s^j)} \quad (8)$$

$$= \rho(t - s) \quad (9)$$

Definition 4.17. An adapted process $X = \{X_t : 0 < t < T\}$ is an *Itô Process* if there exists stochastic processes $\mu = \{\mu_t : 0 < t < T\}$ and $\sigma = \{\sigma_t : 0 < t < T\}$ such that

$$X_t = X_0 + \int_0^t \mu_s ds + \int_0^t \sigma_s dW_s \quad (10)$$

often written in differential notation as

$$dX_t = \mu_t ds + \sigma_t dW_t \quad (11)$$

where μ is known as the *drift* and σ the *diffusion coefficient* of X .

Proposition 2. Let W be a brownian motion. We state the following identities without proof:

1. $(dt)^2 = 0$
2. $dt dW = 0$
3. $(dW)^2 = dt$

4.5.1 Itô's Lemma

Let $f : [0, \infty) \times \mathbb{R} \rightarrow \mathbb{R}$ be a function s.t $\frac{\partial f}{\partial t}$, $\frac{\partial f}{\partial x^i}$ and $\frac{\partial^2 f}{\partial x^i \partial x^j}$ all exist and are continuous for every $i, j \in \{1, \dots, d\}$. Furthermore, suppose that each X^i for $i \in \{1, \dots, d\}$ is an Itô process satisfying

$$dX_t^i = \mu_t^i dt + \sigma_t^i dW_t^i, \quad (4.5.12)$$

then

$$dY_t = df(t, X_t) = \frac{\partial f}{\partial t} dt + \sum_{i=1}^d \frac{\partial f}{\partial X_t^i} dX_t^i + \frac{1}{2} \sum_{i=1}^d \sum_{j=1}^d \frac{\partial^2 f}{\partial X_t^i \partial X_t^j} (dX_t^i)(dX_t^j) \quad (4.5.13)$$

4.6 Example: Geometric Brownian Motion

Consider the following SDE:

$$dX_t = \mu X_t dt + \sigma X_t dW_t, \quad \mu \in \mathbb{R}, \sigma \in (0, \infty)$$

We would like to solve this explicitly by finding X_t in terms of BM W_t . Applying Itô's Lemma to $Y_t = f(t, X_t) = \ln(X_t)$ and noting that

$$\frac{\partial f}{\partial t} = 0, \quad \frac{\partial f}{\partial X_t} = \frac{1}{X_t}, \quad \frac{\partial^2 f}{\partial X_t^2} = -\frac{1}{X_t^2} \quad (4.6.1)$$

gives

$$dY_t = \frac{1}{X_t} dX_t - \frac{1}{2X_t^2} (dX_t)^2 \quad (4.6.2)$$

$$= \frac{1}{X_t} (\mu X_t dt + \sigma X_t dW_t) - \frac{1}{2X_t^2} (\mu X_t dt + \sigma X_t dW_t)^2 \quad (4.6.3)$$

Expanding, using ?? for simplification, and grouping yields:

$$dY_t = \left(\mu - \frac{1}{2}\sigma^2 \right) dt + \sigma dW_t. \quad (4.6.4)$$

Integrating both sides, we obtain:

$$Y_t = Y_0 + \left(\mu - \frac{1}{2}\sigma^2 \right) t + \sigma W_t \quad (4.6.5)$$

$$\Rightarrow Y_t \sim \mathcal{N} \left(\left(Y_0 + \mu - \frac{1}{2}\sigma^2 \right), \sigma^2 t \right) \quad (4.6.6)$$

Finally, recalling that $Y_t = \log(X_t)$, exponentiating ?? gives:

$$X_t = X_0 \exp \left(\left(\mu - \frac{1}{2}\sigma^2 \right) t + \sigma W_t \right) \quad (4.6.7)$$

which shows that X_t follows a log-normal distribution.

4.7 Option Pricing

Definition 4.18. Let $(\Omega, \mathcal{F}, \mathbb{F}, \mathbb{P})$ be a filtered probability space and let H_t represent the price of some contingent claim H at time t . We define this price to be

$$H_t = \mathbb{E}[H_T | \mathcal{F}_t]$$

4.7.1 Pricing under Black-Scholes

Let $(\Omega, \mathcal{F}, \mathbb{F}, \mathbb{P})$ be a filtered probability space. Consider a contingent claim of the form

$$H_T = \max \{X_T^1, X_T^2\} = X_T^1 \mathbb{I}_{\{X_T^1 \geq X_T^2\}} + X_T^2 \mathbb{I}_{\{X_T^2 \geq X_T^1\}}. \quad (4.7.1)$$

For notational ease set: $X_t^{\frac{1}{2}} = \frac{X_t^1}{X_t^2}$, then ?? becomes:

$$H_T = X_T^1 \mathbb{I}_{\{1 \geq X_T^{\frac{2}{1}}\}} + X_T^2 \mathbb{I}_{\{1 \geq X_T^{\frac{1}{2}}\}} \quad (4.7.2)$$

Using definition ?? we have that

$$H_t = \mathbb{E}[H_T | \mathcal{F}_t] \quad (4.7.3)$$

$$= \mathbb{E} \left[X_T^1 \mathbb{I}_{\{1 \geq X_T^{\frac{2}{1}}\}} + X_T^2 \mathbb{I}_{\{1 \geq X_T^{\frac{1}{2}}\}} \middle| \mathcal{F}_t \right] \quad (4.7.4)$$

$$= \mathbb{E} \left[X_T^1 \mathbb{I}_{\{1 \geq X_T^{\frac{2}{1}}\}} \middle| \mathcal{F}_t \right] + \mathbb{E} \left[X_T^2 \mathbb{I}_{\{1 \geq X_T^{\frac{1}{2}}\}} \middle| \mathcal{F}_t \right] \quad (4.7.5)$$

$$= \mathbb{E}^1 \left[\frac{X_t^1}{X_t^1} X_T^1 \mathbb{I}_{\{1 \geq X_T^{\frac{2}{1}}\}} \middle| \mathcal{F}_t \right] + \mathbb{E}^2 \left[\frac{X_t^2}{X_t^2} X_T^2 \mathbb{I}_{\{1 \geq X_T^{\frac{1}{2}}\}} \middle| \mathcal{F}_t \right] \quad (4.7.6)$$

$$= X_t^1 \mathbb{E}^1 \left[\mathbb{I}_{\{1 \geq X_T^{\frac{2}{1}}\}} \middle| \mathcal{F}_t \right] + X_t^2 \mathbb{E}^2 \left[\mathbb{I}_{\{1 \geq X_T^{\frac{1}{2}}\}} \middle| \mathcal{F}_t \right] \quad (4.7.7)$$

$$= X_t^1 \mathbb{Q}^1 \left(\left\{ 1 \geq X_T^{\frac{2}{1}} \right\} \middle| \mathcal{F}_t \right) + X_t^2 \mathbb{Q}^2 \left(\left\{ 1 \geq X_T^{\frac{1}{2}} \right\} \middle| \mathcal{F}_t \right) \quad (4.7.8)$$

Thus if we can evaluate the probabilities $\mathbb{Q}^1$ and $\mathbb{Q}^2$ then not only can we calculate the price of the contingent claim at time t , but these probabilities would also give exactly the portfolio holdings of X_t^1 and X_t^2 needed to perfectly replicate the value of the claim H (if we could trade continuously).

Now suppose X_t^1 and X_t^2 represent the price of two stocks as described under the Black-Scholes model:

$$dX_t^i = \mu^i X_t^i dt + \sigma^i X_t^i dW_t^i, \quad \mu \in \mathbb{R}, \sigma \in (0, \infty) \quad i \in \{1, 2\}. \quad (4.7.9)$$

We wish to find the distribution of $X_t^{\frac{i}{j}}$ in order to evaluate the probability in ??.

Setting $Y_t = f(t, X_t^1, X_t^2) = X_t^{\frac{i}{j}}$, applying ??, and subsequently subbing in ?? we find

$$dY_t = 0 + \frac{1}{X_t^j} dX_t^i - \frac{X_t^i}{(X_t^j)^2} dX_t^j + \frac{1}{2} \left(0(dX_t^i)^2 + \frac{2X_t^i}{(X_t^j)^3} (dX_t^j)^2 - \frac{2}{(X_t^j)^2} (dX_t^i)(dX_t^j) \right) \quad (4.7.10)$$

$$= \frac{1}{X_t^j} (\mu^i X_t^i dt + \sigma^i X_t^i dW_t^i) - \frac{X_t^i}{(X_t^j)^2} (\mu^j X_t^j dt + \sigma^j X_t^j dW_t^j) + \frac{X_t^i}{(X_t^j)^3} (\sigma_j^2 (X_t^j)^2 dt) - \frac{1}{(X_t^j)^2} (\sigma_i \sigma_j X_t^i X_t^j \rho dt) \quad (4.7.11)$$

$$= X_t^{\frac{i}{j}} \left[(\mu^i - \mu^j + \sigma_j^2 - \sigma_i \sigma_j \rho) dt + \sigma_i dW_t^i - \sigma_j dW_t^j \right] \quad (4.7.12)$$

Under $\mathbb{Q}^i$ the drift goes to zero and we are simply left with:

$$dX_t^{\frac{i}{j}} = X_t^{\frac{i}{j}} \left[\sigma_i dW_t^i - \sigma_j dW_t^j \right] \quad (4.7.13)$$

This looks promising, as it resembles the Black-Scholes formulation for a single stock, however we need to do some further manipulation to get it relative to a single BM process.

For notational simplification, set $X_t = X_t^{\frac{i}{j}}$.

Setting $Y_t = \ln(X_t)$, applying Itos Lemma, substituting ?? and finally simplifying using ?? yields:

$$dY_t = \frac{1}{X_t} dX_t - \frac{1}{2(X_t)^2} (dX_t)^2 \quad (4.7.14)$$

$$= \sigma_i dW_t^i - \sigma_j dW_t^j - \frac{1}{2} (\sigma_i^2 + \sigma_j^2 - 2\sigma_i \sigma_j \rho) dt \quad (4.7.15)$$

Set $(\sigma^*)^2 = \sigma_i^2 + \sigma_j^2 - 2\sigma_i \sigma_j \rho$

Claim 1. We claim that $W_t = \frac{\sigma_i W_t^i - \sigma_j W_t^j}{\sigma^*}$ is a BM.

Proof. 1. Clearly $W_0 = 0$ a.s since $W_0^i = W_0^j = 0$ a.s.

2. Similarly, W_t has stationary and independent increments.

3. Noting that $W_t - W_s = \frac{1}{\sigma^*} (\sigma_i (W_t^i - W_s^i) + \sigma_j (W_t^j - W_s^j))$

$$Var(W_t - W_s) = \frac{1}{(\sigma^*)^2} \left[\sigma_i^2 (t-s) + \sigma_j^2 (t-s) + 2\sigma_i \sigma_j Cov(\sigma_i (W_t^i - W_s^i), W_t^j - W_s^j) \right] \quad (4.7.16)$$

$$= \frac{1}{(\sigma^*)^2} [(t-s)(\sigma_i^2 + \sigma_j^2 + \sigma_i \sigma_j \rho)] \quad (4.7.17)$$

$$= (t-s) \quad (4.7.18)$$

□

Thus we have

$$dY_t = -\frac{1}{2}(\sigma^*)^2 dt + \sigma^* dW_t \quad (4.7.19)$$

$$\Rightarrow \ln(X_T^{\frac{i}{j}}) = Y_T = Y_t + \int_t^T -\frac{1}{2}(\sigma^*)^2 ds + \int_t^T \sigma^* dW_s \quad (4.7.20)$$

$$= Y_t - \frac{1}{2}(\sigma^*)^2(T-t) + \sigma^*(W_T - W_t) \quad (4.7.21)$$

$$\Rightarrow X_T = X_t \exp \left(-\frac{1}{2}(\sigma^*)^2(T-t) + \sigma^*(W_T - W_t) \right) \quad (4.7.22)$$

Returning to ?? we find

$$\mathbb{Q}^i \left(\left\{ 1 \geq X_T^{\frac{i}{j}} \right\} | \mathcal{F}_t \right) = \mathbb{Q}^i \left(\left\{ 0 \geq \ln(X_T^{\frac{i}{j}}) \right\} | \mathcal{F}_t \right) \quad (4.7.23)$$

$$= \mathbb{Q}^i \left(\left\{ 0 \geq \ln(X_t^{\frac{i}{j}}) - \frac{1}{2}(\sigma^*)^2(T-t) + \sigma^*(W_T - W_t) \right\} | \mathcal{F}_t \right) \quad (4.7.24)$$

$$= \mathbb{Q}^i \left(\left\{ \frac{W_T - W_t}{\sqrt{T-t}} \leq \frac{\ln X_t^{\frac{i}{j}} + \frac{1}{2}(\sigma^*)^2(T-t)}{\sigma^* \sqrt{(T-t)}} \right\} | \mathcal{F}_t \right) \quad (4.7.25)$$

$$= \Phi(d_1^{\frac{i}{j}}(t)) \quad (4.7.26)$$

Noting that

$$d_2^{\frac{i}{j}}(t) = d_1^{\frac{i}{j}}(t) - \sigma \sqrt{T-t} = \frac{\ln X_t^{\frac{i}{j}} - \frac{1}{2}(\sigma^*)^2(T-t)}{\sigma^* \sqrt{(T-t)}}$$

we find that $d_1^{\frac{i}{j}}(t) = -d_2^{\frac{i}{j}}(t)$.

Thus, our final formula for H_t is

$$H_t = X_t^1 \Phi \left(d_1^{\frac{i}{j}}(t) \right) + \Phi \left(-d_2^{\frac{i}{j}}(t) \right) \quad (4.7.27)$$

4.7.2 Estimating σ_i, σ_j and ρ

In order to calculate the holdings in each stock as described above, we need to estimate σ^* for which we need estimates of σ_i, σ_j and ρ . The derivation below motivates our choice of estimator.

Let S_t be the price of our stock at time t and suppose our most recent observation of the stock is at time $t - dt$ (in our specific case $dt = \frac{1}{365}$, i.e. one day). By Black-Scholes we have that

$$S_t = S_{t-dt} \exp \left[\left(\mu - \frac{1}{2}\sigma^2 \right) dt + \sigma (W_t - W_{t-dt}) \right] \quad (4.7.28)$$

$$\Rightarrow \ln \left(\frac{S_t}{S_{t-dt}} \right) = \left(\mu - \frac{1}{2}\sigma^2 \right) dt + \sigma (W_t - W_{t-dt}) \quad (4.7.29)$$

From this we see that for no matter which t is chosen, $\ln\left(\frac{S_t}{S_{t-dt}}\right)$ is identically and independently distributed as long as we consider disjoint intervals. Taking expectations on both sides and recalling that increments of BM have zero mean:

$$\mathbb{E}\left[\ln\left(\frac{S_t}{S_{t-dt}}\right)\right] = \left(\mu - \frac{1}{2}\sigma^2\right)dt \quad (4.7.30)$$

$$(4.7.31)$$

from which we find that

$$TS2 = \ln\left(\frac{S_t}{S_{t-dt}}\right) - \mathbb{E}\left[\ln\left(\frac{S_t}{S_{t-dt}}\right)\right] = \sigma(W_t - W_{t-dt}) \quad (4.7.32)$$

and consequently that

$$Var(TS2) = \sigma^2(dt). \quad (4.7.33)$$

Since we can get unbiased estimators for the LHS of ?? by simply taking the mean of all log ratio increments, we get our point estimate for σ as

$$\sigma = \sqrt{Var(TS2)/dt}$$

Doing this calculation for each time series related to S_t^i and S_t^j gives point estimates for σ_i and σ_j , and recalling ?? we have that

$$\rho = \frac{Cov(TS2^i, TS2^j)}{\sigma_i \sigma_j dt}$$

which gives us a point estimate for ρ and consequently for σ^* .

4.7.3 Our claim

The claim we will be hedging against is of the form

$$H = 100H^*$$

where

$$H^* = \max\left\{\frac{S_T^1}{S_0^1}, \frac{S_T^2}{S_0^2}\right\}.$$

Note that the derivation of H_t^* for all $t \in \{0, \dots, T\}$ follows identically that of $H_T = \max\{X_T^1, X_T^2\}$ as described above, with X_t^i simply being replaced with $\frac{S_t^i}{S_0^i}$ in all positions.

In other words the option gives you the maximum percentage gain of the stocks relative to their initial prices.

4.8 Problem Formulation

Suppose there is a (potentially unattainable) contingent claim, H , which we would like to hedge. The goal is to be able to track the price of H with minimal risk by making investments in a portfolio. More formally, let $V_T(\varphi)$ be the value of our portfolio at time T under the trading strategy φ , then we to find the optimal strategy

$$\varphi = \arg \min_{\varphi} \mathbb{E} [(H - V_T(\varphi))^2] .$$

The specific claim we will be considering is

$$H_T = \max \left\{ \frac{S_T^1}{S_0^1}, \frac{S_T^2}{S_0^2} \right\} .$$

Note that the derivation for the price of the claim at time t , H_t , will be exactly as found in , except that we replace X_t^i with $\frac{S_t^i}{S_0^i}$ in all positions (including within $d^i(t)$).

4.9 Neural Networks

Neural networks are a ubiquitous architecture in the machine learning field. They act as non-linear function approximators and have been employed to solve a plethora of tasks. They have the ability to outperform humans in complex games (such as chess and go), can be used for speech recognition, image classification, facial recognition as well as the powerful and famous generative uses such as diffusion based image generation and transformer based text generation. Their architectures are relatively simple, yet have shown to

4.9.1 Basic feed-forward MLP Architecture

Suppose we have input/explanatory data, $\mathbf{x} \in \mathbb{R}^d$ and output/response data, $\mathbf{y} \in \mathbb{R}^e$. Let $f : \mathbb{R}^d \rightarrow \mathbb{R}^e$ be the function relating the two, ie: $f(\mathbf{x}) = \mathbf{y}$. A neural network $f^{\mathcal{NN}} : \mathbb{R}^d \rightarrow \mathbb{R}^e$ is a highly parameterised function which approximates f . More formally we have that

$$f^{\mathcal{NN}} = l^{out} \circ l^m \circ \dots \circ l^1 \circ l^{in}$$

where

$$l^j = \sigma \left(\mathbf{w}^{(j-1,j)} \mathbf{k}^j + \mathbf{b}^j \right)$$

with

$$\mathbf{k}^j = l^{j-1} \circ \dots \circ l^1 \circ l^{in}(\mathbf{x}).$$

4.9.2 Universal Approximation Theorem

A major motivation for the use of neural networks is the fact that they are universal approximators. More specifically, it has been shown that a single layer neural network of sufficient depth can arbitrarily approximate any continuous function on some cube. The proof presented here follows that of Cybenko 1989, but with some added details and preliminary results upon which the proof depends.

4.10 Variational Auto-Encoder

4.11 Auto-Encoders

Auto-Encoders are a class of neural networks which can be used for amongst other things: dimensionality reduction, feature extraction and data compression. They combine an encoder and decoder and attempt to reproduce input data by compressing it down into a lower dimensional latent space via the encoder, and then use this latent representation to reproduce the original input via the decoder. In this way the encoder is forced to extract core features of the data so that the decoder can effectively recover the data from the latent encoding.

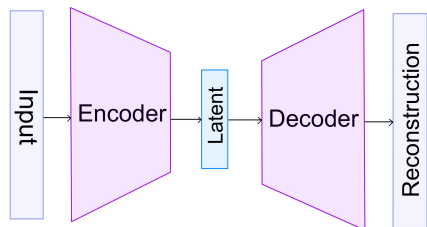


Figure 1: Auto-Encoder Architecture.

This notion of a vectorized latent embedding which captures core information raises interesting questions. Can we take two data samples, get their latent embeddings, linearly interpolate their embeddings and then run this interpolation through the decoder to get a sample which somehow combines the core features of both sample? Can we somehow sample from the latent space to generate wholly novel samples which come from the same distribution as our original data? If we were attempt to do either of these things using a standard Auto-Encoder we would often find that an issue is the lack of regularity of the latent space. What we mean by this is that there is nothing in the architecture of an Auto-Encoder which, for instance, ensures that the latent space surrounding a particular latent embedding will produce samples "similar" to that latent embedding, or that there is any inherent regular distributional structure to the latent space. Although the Auto-Encoder often cluster similar inputs to similar latent embeddings naturally in order for efficient reproductions, the lack of an explicit enforcing of latent space regularity leads to the issue of i) not having a distribution to sample from and ii) not being able to ensure that the latent space surrounding a specific embedding holds "similar" embeddings, ie: a lack of regularity. These issues are addressed in the modified VAE architecture as described below.

4.11.1 General Description

Variational Auto-Encoders (VAEs) are a class of generative neural networks which consist of an encoder and decoder. The encoder maps the input data into a latent space, typically of lower dimension than that of the input, and the decoder maps latent-embeddings back into the "real-world" space. The encoder can be interpreted as applying a non-linear dimensionality reduction to the input data. The aim of the encoder is to learn a useful latent-representation of the data, where core/informative features are captured in the dimensions of the latent space. A key feature of VAEs over their cousin the Auto-Encoder

is that VAEs attempt to force the latent space to be more regular. This can be achieved by penalising the estimated latent space to ensure that it is closer to some prior distribution of our choice. This allows us to sample from the continuous prior after training and consequently pass our samples through the decoder to generate data samples that come from the same distribution as our original data.

4.11.2 Formal Description

The formal description that follows is closely based on the work of Kingma and Welling 2019.

Let $\mathbf{x}$ be a vector representing the k -dimensional data whose joint distribution we would like to model. Let $p^*(\mathbf{x})$ denote the true distribution of X . Our goal is to approximate $p^*(\mathbf{x})$ with a parameterised model $p_{\boldsymbol{\theta}}(\mathbf{x})$. In other words, we wish to find parameters $\boldsymbol{\theta}$ such that

$$\mathbf{x} \sim p^*(\mathbf{x}) \approx p_{\boldsymbol{\theta}}(\mathbf{x}).$$

It holds that we would like the family of models possible under our parameterisation to be highly flexible, as we do not know how complex the true distribution $p^*(\mathbf{x})$ may be. Neural Networks give such a family of models.

We can then introduce the notion of *latent variables*, denoted $\mathbf{z}$, also known as unobservable variables due to the fact that they are not found explicitly in our dataset. Although they are unobserved, they appear in our model through $\mathbf{x}$'s conditional dependence on them. Their distribution, $p(\mathbf{z})$, is known as the prior distribution over $\mathbf{z}$. Via the law of total probability and Bayes' formula we get

$$p_{\boldsymbol{\theta}}(\mathbf{x}) = \int p_{\boldsymbol{\theta}}(\mathbf{x}, \mathbf{z}) d\mathbf{z} \quad (4.1)$$

$$(4.2)$$

and

$$p_{\boldsymbol{\theta}}(\mathbf{x}, \mathbf{z}) = p_{\boldsymbol{\theta}}(\mathbf{x}|\mathbf{z})p(\mathbf{z}) \quad (4.3)$$

When such models are parameterised using deep neural networks they are termed *deep latent variable models* (DLVM). A benefit of DLVM's is that even when each factor in ?? (namely the conditional and prior distributions) are relatively simple (for example, Gaussian), the resulting marginal $p_{\boldsymbol{\theta}}(\mathbf{x})$ can be very complex as a result of ??. This is what makes DLVM's attractive when approximating possibly complex distributions.

When attempting to maximise the likelihood of DVLM's we run into the issue that the marginal $p_{\theta}(\mathbf{x})$ is typically intractable due to the integral in ?? either not having an analytic solution, or being computationally intractable under estimation. As a consequence we cannot differentiate with regards to (w.r.t) the parameters so as to be able to maximise the likelihood. This intractability also carries over to the posterior distribution $p_{\theta}(\mathbf{z}|\mathbf{x})$ via

$$p_{\theta}(\mathbf{z}|\mathbf{x}) = \frac{p_{\theta}(\mathbf{x}, \mathbf{z})}{p_{\theta}(\mathbf{x})}. \quad (4.4)$$

In order to address these issues of intractability we introduce a parameterised, approximate posterior

$$q_{\phi}(\mathbf{z}|\mathbf{x}) \approx p_{\theta}(\mathbf{z}|\mathbf{x}) \quad (4.5)$$

4.11.3 KL-Divergence

Although there are many methods to penalise the "distance" between two distributions, historically Kullback-Leibler(KL) divergence has been used in the context of VAEs in order to get such a measure of "distance", motivated below in the derivation regarding the evidence lower bound. It must be noted that KL divergence is not a true metric in the sense that it fails to be symmetric.

Formally, KL-divergence is the expected difference in information of one probability distribution, q to another distribution p , where the expectation is taken under q . Mathematically it is defined as

$$D_{KL}(q||p) = \mathbb{E}_{q(\mathbf{x})} \left[\log \left(\frac{q(\mathbf{x})}{p(\mathbf{x})} \right) \right] \quad (4.6)$$

The definition stems from information theory, and can be informally interpreted as how surprised one is from using p as a distribution to describe the process in question when the true distribution of the process is q contributors 2024.

Recall Jensen's inequality, namely that if $(\Omega, \mathcal{F}, \mu)$ is a measure space, $f : \Omega \rightarrow \mathbb{R}$ a measurable function, and $\varphi : \mathbb{R} \rightarrow \mathbb{R}$ a convex function, then

$$\varphi \left(\int_{\Omega} f d\mu \right) \leq \int_{\Omega} \varphi(f) d\mu. \quad (4.7)$$

If φ is a strictly convex function and f is non-degenerative (i.e.: non-constant) on a set A with $\mu(A) > 0$, then the inequality is strict.

Claim 2. KL divergence is non-negative.

Proof. We have that $-\log : \mathbb{R} \rightarrow \mathbb{R}$ is a convex function, hence by letting $f = \frac{p(\mathbf{x})}{q(\mathbf{x})}$ we get

$$D_{KL}(q||p) = \mathbb{E}_{q(\mathbf{x})} \left[-\log \left(\frac{p(\mathbf{x})}{q(\mathbf{x})} \right) \right] \quad (4.8)$$

$$\geq -\log \int_{\mathbb{R}} q(\mathbf{x}) \frac{p(\mathbf{x})}{q(\mathbf{x})} dx \quad (4.9)$$

$$= -\log \int_{\mathbb{R}} p(\mathbf{x}) dx \quad (4.10)$$

$$= -\log(1) = 0 \quad (4.11)$$

□

Claim 3. $D_{KL}(q||p) = 0$ if and only if (iff) $p = q$ $\mathbb{P}_{q(\mathbf{x})}$ almost everywhere (a.e).

Proof. ($\Rightarrow$) Suppose that $D_{KL}(q||p) = 0$, then since $-\log$ is a strictly convex function, we can only have equality in ?? if $-\log \frac{p(\mathbf{x})}{q(\mathbf{x})}$ is degenerative everywhere except on a set of measure zero, ie:

$$\begin{aligned} D_{KL}(q||p) &= 0 \\ \Rightarrow p(\mathbf{x}) &= q(\mathbf{x}) \quad \mathbb{P}_{q(\mathbf{x})} \quad a.e. \end{aligned}$$

($\Leftarrow$)

Suppose that $p = q$ $\mathbb{P}_{q(\mathbf{x})}$ a.e, then

$$\begin{aligned} D_{KL}(q||p) &= \mathbb{E}_{q(\mathbf{x})} \log \left(\frac{q(\mathbf{x})}{p(\mathbf{x})} \right) \\ &= \int_{\Omega} \log \left(\frac{q(\mathbf{x})}{p(\mathbf{x})} \right) \mathbb{I}_{\{q(\mathbf{x})=p(\mathbf{x})\}} d\mathbb{P}_{q(\mathbf{x})} \\ &= \int_{\{q(\mathbf{x})=p(\mathbf{x})\}} (\log(1)) d\mathbb{P}_{q(\mathbf{x})} \\ &= 0. \end{aligned}$$

□

4.11.4 Evidence lower bound(ELBO)

As with many problems of estimating the true distribution of a process, $p(\mathbf{x})$, we wish to maximise the likelihood of our data being observed under our approximate, parameterised distribution, $p_{\theta}(\mathbf{x})$. The evidence/variational lower bound provides a lower bound for the log-likelihood of our estimate. Let $q_{\phi}(\mathbf{z}|\mathbf{x})$

denote our inference model/decoder, $p_{\theta}(\mathbf{z}|\mathbf{x})$ be the true but intractable posterior and let $p_{\theta}(\mathbf{x})$ the marginal distribution of $\mathbf{X}$ under our parameterisation. We have that

$$\log p_{\theta}(\mathbf{x}) = \mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x})} [\log p_{\theta}(\mathbf{x})] \quad (4.12)$$

$$= \mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x})} \left[\log \left(\frac{p_{\theta}(\mathbf{x}, \mathbf{z})}{p_{\theta}(\mathbf{z}|\mathbf{x})} \right) \right] \quad (4.13)$$

$$= \mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x})} \left[\log \left(\frac{p_{\theta}(\mathbf{x}, \mathbf{z}) q_{\phi}(\mathbf{z}|\mathbf{x})}{q_{\phi}(\mathbf{z}|\mathbf{x}) p_{\theta}(\mathbf{z}|\mathbf{x})} \right) \right] \quad (4.14)$$

$$= \mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x})} \left[\log \left(\frac{p_{\theta}(\mathbf{x}, \mathbf{z})}{q_{\phi}(\mathbf{z}|\mathbf{x})} \right) \right] + \mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x})} \left[\log \left(\frac{q_{\phi}(\mathbf{z}|\mathbf{x})}{p_{\theta}(\mathbf{z}|\mathbf{x})} \right) \right] \quad (4.15)$$

$$= \mathcal{L}_{\theta, \phi}(\mathbf{x}) + D_{KL}(q_{\phi}(\mathbf{z}|\mathbf{x}) || p_{\theta}(\mathbf{z}|\mathbf{x})) \quad (4.16)$$

The first term is called the variational/evidence lower bound (ELBO), denoted $\mathcal{L}_{\theta, \phi}(\mathbf{x})$, and the second term is the KL-divergence between $q_{\phi}(\mathbf{z}|\mathbf{x})$ and $p_{\theta}(\mathbf{z}|\mathbf{x})$, denoted $D_{KL}(q_{\phi}(\mathbf{z}|\mathbf{x}) || p_{\theta}(\mathbf{z}|\mathbf{x}))$.

Since the KL-divergence is always non-negative, we see that the evidence lower bound is indeed a lower bound for the log-likelihood, ie:

$$\log p_{\theta}(\mathbf{x}) \geq \mathcal{L}_{\theta, \phi}(\mathbf{x}).$$

Rearranging gives

$$\mathcal{L}_{\theta, \phi}(\mathbf{x}) = \log p_{\theta}(\mathbf{x}) - D_{KL}(q_{\phi}(\mathbf{z}|\mathbf{x}) || p_{\theta}(\mathbf{z}|\mathbf{x}))$$

which has a very satisfying interpretation. We see that by maximising the ELBO, we not only approximately maximise the marginal log-likelihood of $\mathbf{X}$, but also minimise the KL divergence between the true and approximated posterior distributions.

Manipulating the ELBO we find:

$$\mathcal{L}_{\theta, \phi}(\mathbf{x}) = \mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x})} \left[\log \left(\frac{p_{\theta}(\mathbf{x}|\mathbf{z}) p(\mathbf{z})}{q_{\phi}(\mathbf{z}|\mathbf{x})} \right) \right] \quad (4.17)$$

$$= \mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x})} [\log (p_{\theta}(\mathbf{x}|\mathbf{z}))] - \mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x})} \left[\log \left(\frac{q_{\phi}(\mathbf{z}|\mathbf{x})}{p_{\theta}(\mathbf{z})} \right) \right] \quad (4.18)$$

$$= \mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x})} [\log (p_{\theta}(\mathbf{x}|\mathbf{z}))] - D_{KL}(q_{\phi}(\mathbf{z}|\mathbf{x}) || p_{\theta}(\mathbf{z})) \quad (4.19)$$

$$= \text{Reconstruction Loss} - \text{KL Divergence} \quad (4.20)$$

The first term in ?? is termed the *reconstruction* loss and the second is the KL divergence between our approximate posterior distribution and our prior distribution.

4.11.5 Deriving the loss function

We now derive an explicit form for the loss function of our neural network, which in our case is the negative ELBO. In order to do this, we must decide on forms for both the prior, posterior and decoder distributions. The most common case, and indeed what we used when implementing the VAE, is to choose the prior and posterior to be Gaussian distributions with diagonal covariance matrices:

$$q_\phi(\mathbf{z}|\mathbf{x}) \sim \mathcal{N}(\mathbf{x}, \boldsymbol{\mu}(\mathbf{x}), \text{diag}(\boldsymbol{\sigma}^2(\mathbf{x}))) \quad (4.21)$$

and

$$p(\mathbf{z}) \sim \mathcal{N}(\mathbf{0}, \mathbb{I}) \quad (4.22)$$

where

$$(\boldsymbol{\mu}(\mathbf{x}), \log(\boldsymbol{\sigma}^2(\mathbf{x}))) = \text{EncoderNeuralNetwork}_\phi(\mathbf{x}) \quad (4.23)$$

Recall that the distribution function for a d -dimensional multivariate gaussian random variable $\mathbf{Y}$ is of the form

$$p_{\mathbf{Y}}(\mathbf{y}) = ((2\pi)^d |\boldsymbol{\Sigma}|)^{-\frac{1}{2}} \exp[(\mathbf{y} - \boldsymbol{\mu})^T \boldsymbol{\Sigma}^{-1} (\mathbf{y} - \boldsymbol{\mu})] \quad (4.24)$$

where $\boldsymbol{\Sigma}$ is the covariance matrix of $\mathbf{Y}$.

Thus the densities for the prior and posterior are

$$p(\mathbf{z}) = (2\pi)^{-\frac{d}{2}} \exp\left(-\frac{1}{2} \sum_{i=1}^d (z_i)^2\right) \quad (4.25)$$

and

$$q_\phi(\mathbf{z}|\mathbf{x}) = ((2\pi)^d |\boldsymbol{\Sigma}|)^{-\frac{1}{2}} \exp\left(-\frac{1}{2} \sum_{i=1}^d \left(\frac{z_i - \mu_i}{\sigma_i}\right)^2\right) \quad (4.26)$$

Focusing on the KL Divergence term in the ELBO ?? and subbing in the chosen densities for the prior and posterior gives:

$$D_{KL}(q_\phi(\mathbf{z}|\mathbf{x})||p_\theta(\mathbf{z})) \quad (4.27)$$

$$= \mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})} \left[\log \left(\frac{q_\phi(\mathbf{z}|\mathbf{x})}{p_\theta(\mathbf{z})} \right) \right] \quad (4.28)$$

$$= \mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})} [\log q_\phi(\mathbf{z}|\mathbf{x}) - \log p(\mathbf{z})] \quad (4.29)$$

$$= \mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})} \left[-\frac{1}{2} \left(\log(2\pi)^d + \log|\boldsymbol{\Sigma}| - \sum_{i=1}^d \left(\frac{z_i - \mu_i}{\sigma_i} \right)^2 + \log(2\pi)^d + \sum_{i=1}^d (z_i)^2 \right) \right] \quad (4.30)$$

$$= -\frac{1}{2} \left[\log(\prod_{i=1}^d \sigma_i) + \sum_{i=1}^d \frac{\mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})}(z_i - \mu_i)^2}{\sigma_i^2} - \sum_{i=1}^d \mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})}(z_i^2) \right]. \quad (4.31)$$

Recalling that $\mathbb{E}(z_i^2) = \sigma_i^2 - \mu_i^2$ and $\text{Var}(z_i) = \mathbb{E}(z_i - \mu_i)^2$, line ?? becomes

$$= -\frac{1}{2} \left[\sum_{i=1}^d (\log \sigma_i^2 + 1 - \sigma_i^2 - \mu_i^2) \right] \quad (4.32)$$

When dealing with continuous data, the form of the decoder's distribution is assumed to be Gaussian distributed, with mean given by the model's output, and with constant, diagonal covariance structure:

$$p_{\theta}(\mathbf{x}|\mathbf{z}) \sim \mathcal{N}(\mathbf{z}, \boldsymbol{\mu}(\mathbf{z}), \boldsymbol{\sigma}^2 \mathbb{I}) \quad (4.33)$$

where

$$\boldsymbol{\mu}(\mathbf{z}) = \text{DecoderNeuralNetwork}_{\theta}(\mathbf{z})$$

Let k be the dimensionality of the input $\mathbf{x}$, then the reconstruction error term in ?? becomes:

$$\mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x})} [\log(p_{\theta}(\mathbf{x}|\mathbf{z}))] = \mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x})} \left[-\frac{1}{2\sigma^2} \sum_{i=1}^k (x_i - \mu(z)_i)^2 - \frac{k}{2} \log(2\pi\sigma^2) \right] \quad (4.34)$$

Thus the loss function which is the negative ELBO becomes:

$$Loss = \mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x})} \left[\frac{1}{2\sigma^2} \sum_{i=1}^k (x_i - \mu(z)_i)^2 - \frac{k}{2} \log(2\pi\sigma^2) \right] - \frac{1}{2} \left[\sum_{i=1}^d (\log \sigma_i^2 + 1 - \sigma_i^2 - \mu_i^2) \right] \quad (4.35)$$

4.11.6 Gradient Estimates

If we wish to use gradient based optimisation techniques such as Stochastic Gradient Descent (SGD) in order to maximise the ELBO (or equivalently, minimise the loss function), we need to be able to find unbiased estimators for the gradient of the ELBO w.r.t both sets of parameters θ and ϕ , i.e. we need to get unbiased estimates for:

$$\nabla_{\theta\phi} \mathcal{L}_{\theta,\phi}(\mathbf{x}) = \nabla_{\theta\phi} (\text{Reconstruction Loss} - \text{KL divergence}) \quad (4.36)$$

$$= \nabla_{\theta\phi} (\text{Reconstruction Loss}) - \nabla_{\theta\phi} (\text{KL divergence}) \quad (4.37)$$

$$(4.38)$$

Finding gradients for the KL divergence loss term is straightforward as the loss depends only on the outputs of the encoder, which in our case is a differentiable function.

Let us now attempt to find the gradient of the Reconstruction Loss.

We first consider taking the gradient w.r.t θ :

$$\nabla_{\theta} (\text{Reconstruction Loss}) = \nabla_{\theta} \mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x})} [\log(p_{\theta}(\mathbf{x}|\mathbf{z}))] \quad (4.39)$$

$$= \mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x})} [\nabla_{\theta} \log(p_{\theta}(\mathbf{x}|\mathbf{z}))] \quad (4.40)$$

$$\approx \nabla_{\theta} \log(p_{\theta}(\mathbf{x}|\mathbf{z})) \quad (4.41)$$

where ?? is an unbiased Monte Carlo estimate of ??, gotten by sampling from $\mathbf{z} \sim q_{\phi}(\mathbf{z}|\mathbf{x})$.

We however run into issues when attempting to take the derivative w.r.t ϕ as we can no longer simply move the derivative inside the integral due to us taking the integral relative to $q_{\phi}(\mathbf{z}|\mathbf{x})$ which depends on ϕ . A work around is if we can somehow express the stochastic element of $q_{\phi}(\mathbf{z}|\mathbf{x})$ by something independent of ϕ , allowing us to differentiate through expectation after a change of variables. This approach is famously known as the reparameterisation trick.

4.11.7 The Reparameterisation Trick

Suppose we can express the random variable $\mathbf{z} \sim q_{\phi}(\mathbf{z}|\mathbf{x})$ as the transformation of another random variable ϵ , ie:

$$\mathbf{z} = g(\phi, \mathbf{x}, \epsilon) \quad (4.42)$$

where ϵ has some distribution independent of ϕ and $\mathbf{x}$, say $\epsilon \sim p(\epsilon)$, and g is an invertible and differentiable function.

We can now express expectations under $q_{\phi}(\mathbf{z}|\mathbf{x})$ as being under $p(\epsilon)$:

$$\mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x})} f(\mathbf{z}) = \mathbb{E}_{p(\epsilon)} f(\mathbf{z}) \quad (4.43)$$

and because we are now taking expectation under a distribution independent of ϕ , the derivative and expectations become commutative as was the case with θ :

$$\nabla_{\phi} (\text{Reconstruction Loss}) = \nabla_{\phi} \mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x})} [\log(p_{\theta}(\mathbf{x}|\mathbf{z}))] \quad (4.44)$$

$$= \nabla_{\phi} \mathbb{E}_{p(\epsilon)} [\log(p_{\theta}(\mathbf{x}|\mathbf{z}))] \quad (4.45)$$

$$= \mathbb{E}_{p(\epsilon)} [\nabla_{\theta} \log(p_{\theta}(\mathbf{x}|\mathbf{z}))] \quad (4.46)$$

$$\approx \nabla_{\theta} \log(p_{\theta}(\mathbf{x}|\mathbf{z})) \quad (4.47)$$

where line ?? is again a Monte Carlo estimator of ??, however samples of $\mathbf{z}$ are now gotten by randomly sampling ϵ and applying the transformation $\mathbf{z} = g(\phi, \mathbf{x}, \epsilon)$.

We are thus now able to take the derivatives of the ELBO w.r.t both the encoder and decoder's parameters, allowing us to use gradient based optimisation techniques to maximise the ELBO.

4.11.8 Final Loss

Revisiting the Reconstruction Loss in ?? and using our new reparameterisation trick to consider the derivative within the expectation, we see that when taking the derivatives w.r.t to θ and ϕ , the second term falls away (as it is independent of both the encoder and decoder's parameters), and the constants in front of the summation can be disregarded, meaning we just need to consider terms of the form:

$$-\sum_{i=1}^n (x_i - \mu(z)_i)^2 \quad (4.48)$$

which is the negative of the familiar Mean Squared Error Loss (MSE).

Thus, by choosing the loss to be the negative ELBO, and using the fact that we will be taking unbiased estimators of the gradient of this loss via sampling under $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbb{I})$, we arrive at the final loss function for the j_{th} sample $\mathbf{x}_j$:

$$Loss_j = \sum_{i=1}^k (x_{ij} - \mu(z)_{ij})^2 - \frac{1}{2} \sum_{i=1}^d (\log \sigma_{ij}^2 + 1 - \sigma_{ij}^2 - \mu_i^2) \quad (4.49)$$

$$= \text{MSE}_{\text{loss}} + \text{KL}_{\text{loss}} \quad (4.50)$$

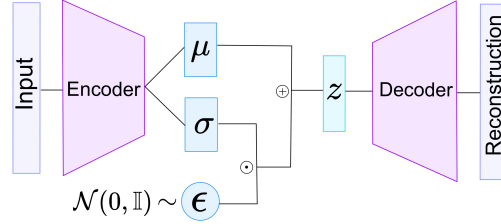


Figure 2: Variational Auto-Encoder Architecture.

4.11.9 Conditional VAEs

A modification of the usual VAE architecture was used in this implementation, known as the Conditional VAE (CVAE). A slight modification allows one to append another input to be conditioned on for both the encoder and decoder networks. In our case this has particular relevance in allowing us to condition on the most recent window of time when predicting the next signature, as the most recent events in time have the most impact on current events. By allowing both the encoder and decoder to take in these conditional inputs, we are also able to condition upon the previously generated signature during market simulation, meaning that generated paths will be more coherent.

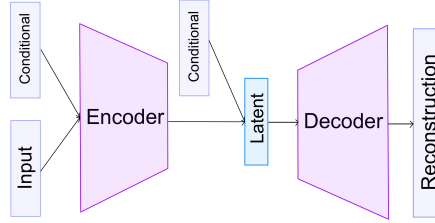


Figure 3: Conditional Variational Auto-Encoder Architecture.

5 Methodology

5.1 Path Generation

5.1.1 Data Collection

The two historical stock prices of Apple Yahoo Finance n.d. and Microsoft Finance n.d. were gotten from the Yahoo! finance website and the Opening price of the stocks for the period of 22/06/2022-17/06/2024, corresponding to 500 days of historical price data.

5.1.2 Data Pre-Processing

The 500 days of historical price data was divided into 100 subpaths of dimension 2 (for the two stocks), each of length 5 days. The log-signature of each of these 100 5 x 2 subpaths was then taken using the iisignature package Graham and Reizenstein 2024, each producing a log-signature of dimension 14. After this, the data was normalised to be in the interval $[0, 1]$ using sklearn MinMaxScaler, which linearly scales down features to be in a fixed range, thus not reducing the effect of outliers. The parameters for the scaler were stored so that they could eventually be used to denormalise the generated samples. These normalised log-signatures were then used as input to train the CVAE.

5.1.3 CVAE

The hyperparameters for the CVAE we informed by those used in Buehler, Horvath, Lyons, Arribas, et al. 2020, with adjustments being made to both the KL weighting and number of nodes in the hidden layers in order to accommodate the difference in data (we simulated two stocks, they only simulated one). The CVAE followed a simple design with three hidden layers: one for the encoder, one for the latent space and one for decoder.

The table below summarises the hyperparameters used:

Hyperparameter	Value
Kl weighting	0.0003
latent dimension	8
width of hidden layers	100
hidden layer activations	LeakyReLU
LeakyReLU negative slope	0.3
output activation	sigmoid
batch size	16
learning rate	0.0003
number of epochs	2000

Table 1: Hyperparameter Configurations

5.1.4 Loss function

With the pre-determined KL weighting of 0.0003, the loss function for the VAE per training example becomes

$$Loss = 0.9997 \times MSE + 0.0003 \times KL_{loss}$$

where the MSE and KL_{loss} terms are calculated as in ??.

5.1.5 Generation/Market Simulation

After training, the marginal distributions of the latent embeddings of the training data were plotted, visualised below:

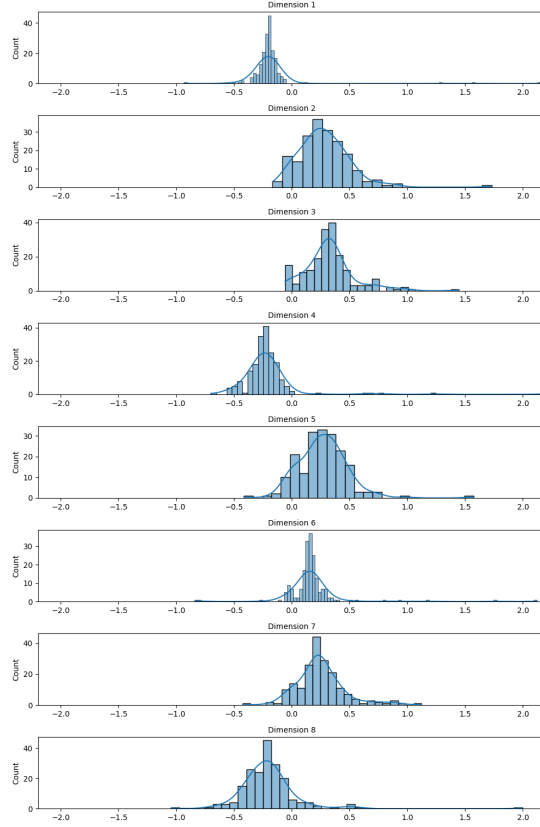


Figure 4: Box plots of latent dimensions.

We note that although the shape of each latent dimension is approximately standard normal, the exact mean and variance varies between dimensions. This is because we lowered the weighting of the KL divergence term, as otherwise the model was over-regularised, causing latent space collapse (i.e. all of the generated samples looked the same).

Standard practice in VAE's is to simply sample from the prior distribution $z \sim \mathcal{N}(\mathbf{0}, \mathbb{I})$ in order to generate novel samples, however, in light of the above latent dimension projections, we decided to sample from a multivariate gaussian $z \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$ where $\boldsymbol{\mu}_i$ is simply the mean of the i_{th} dimension's latent embeddings, and $\boldsymbol{\Sigma}$ is a diagonal covariance matrix where $\boldsymbol{\Sigma}_{ii}$ is the variance of the i_{th} dimension's latent embeddings.

After the CVAE was trained, 100 000, 30 day paths were simulated. This was done by using the CVAE to generate 6 5-day log-signatures, each conditioned on the previous time-period's log-signature. The pseudo-code below describes the exact methodology followed:

Algorithm 1 Market Simulation

```

1: Initialize signature_paths as an empty list
2: for  $i = 1$  to 100,000 do
3:   conditional  $\leftarrow$  log-signature of the last path in training data
4:   path  $\leftarrow$  an empty list
5:   for  $j = 1$  to 6 do
6:     Sample  $z \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$ 
7:      $\mathbf{x} \leftarrow \text{CVAE\_Decoder}(z, \text{conditional})$ 
8:     conditional  $\leftarrow \mathbf{x}$ 
9:     Append  $\mathbf{x}$  to path
10:  end for
11:  Append path to signature_paths
12: end for

```

5.2 Deep Hedging

5.2.1 Data

The data used to train the Deep Hedger is of course the paths generated using the CVAE. 100 000 paths were simulated and this data was randomly split into training, validation and testing datasets using a 70/20/10 split.

5.2.2 Neural Network Architecture

A simple feedforward network was employed for the hedging problem. The network takes in as input the two stock's current prices and the time to maturation of the option, ie:

$$\text{input} = [S_t^1, S_t^2, T - t]$$

and outputs ϕ , the holdings of the two stocks in the portfolio at time t , ie:

$$\text{output} = \phi_t = [\phi_t^1, \phi_t^2].$$

5.2.3 Loss function

Prior to training, the option prices $H_i = [H_{i,1}, H_{i,2}, \dots, H_{i,T}]$ was calculated for each input path $1 \leq i \leq 100000$ as informed by the Black-Scholes equation as described in ??, where $H_{i,T} = 100 \times \max \left\{ \frac{S_T^1}{S_0^1}, \frac{S_T^2}{S_0^2} \right\}$ is the actual payoff of the i_{th} path. For each input path, the loss was calculated to be the sum of squared error between the value of our portfolio and the options price, namely:

$$\text{loss} = \sum_{t=1}^T (V_{i,t} - H_{i,t})^2$$

where $V_{i,t} = V_{i,0} + \sum_{j=1}^t (\Delta S \cdot \phi_j)$ is the value of our portfolio at time t as governed by the trading strategy given by our neural network.

5.2.4 Hyperparameter Search

Optuna Akiba et al. 2019 was used in order to find the optimal set of hyperparameters for the model. The hyperparameters, their range, as well as the optimal values found are presented in the table below:

Hyperparameter	Range/Values Considered	Optimal Value Found
activation	{ReLU, Tanh, LeakyReLU}	
learning rate	[1e-5, 1e-2]	
batch size	{8, 64, 128, 512, 1024, 2048, 4096}	
optimizer	{Adam, SGD, RMSProp}	
l1 regularisation	[0, 1e-2]	
l2 regularisation	[0, 1e-2]	
number of layers	[1, 5]	
width layer 1	[32, 512]	
dropout layer 1	[0, 0.5]	

Table 2: Hyperparameter search space and optimal hyperparameters found

6 Results

6.1 Path Generation

The plots below shows the Mean Squared Error (MSE), KL-loss, and total loss of the VAE over the 15 000 epochs of training.

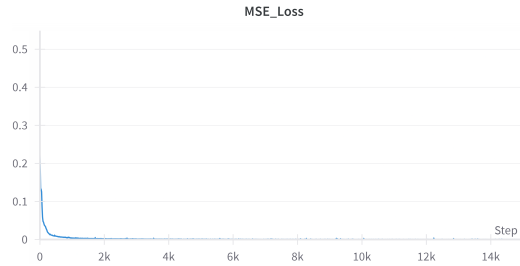


Figure 5: MSE loss

We note that the MSE loss quickly saturates, and the only thing really changing during later epochs is the slow decrease in KL divergence loss as the neural network attempts to coerce the latent space’s distribution to more closely resemble that of a standard Gaussian.

Below is a plot of the full historical path data used for training, as well as all 100 000 generated paths.

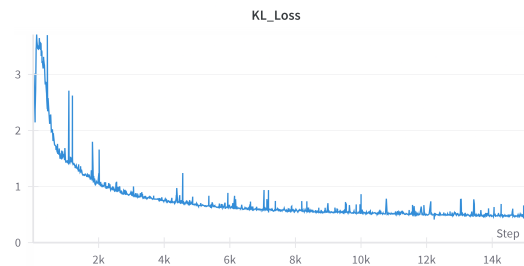


Figure 6: KL divergence loss

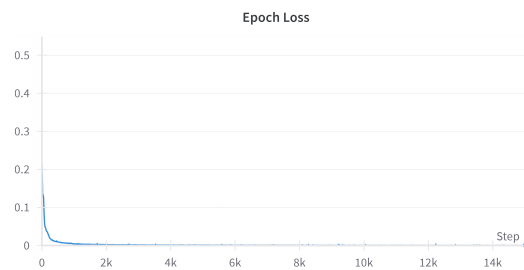


Figure 7: Total loss

The plots below show 1000 generated samples for both the Apple and Microsoft stock prices.

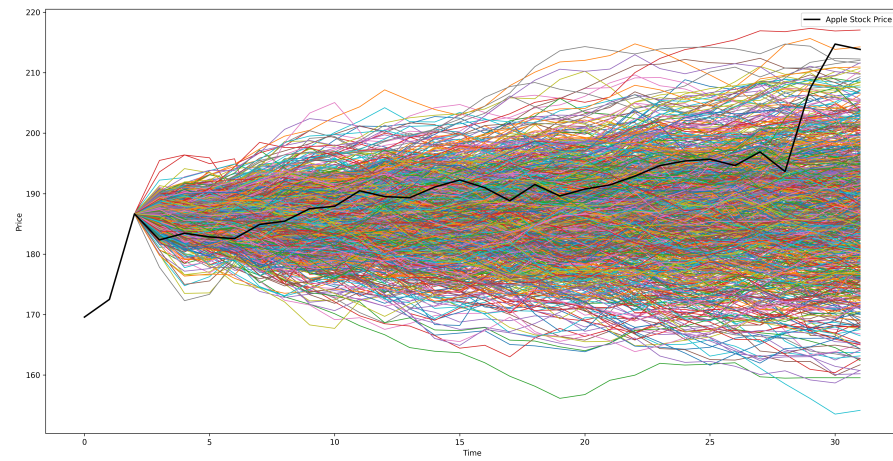


Figure 8: Original and Generated Apple Stock paths

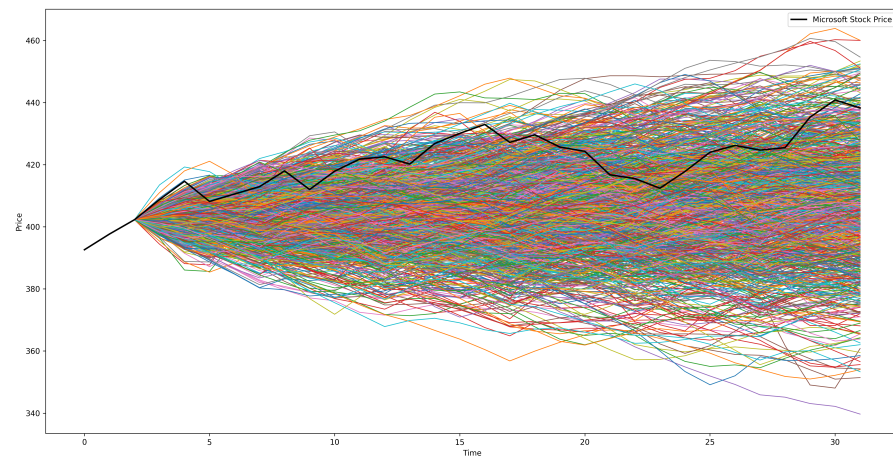


Figure 9: Original and Generated Microsoft Stock paths

6.2 Deep Hedging

7 Discussion

With regards to the DNN informed strategy, we see that our findings agree with those of Stangroom Stangroom 2023 and Buehler et al. Buehler, Gonon, et al. 2019 in that the DNN strategy outperformed more traditionally informed strategies when hedging a European style basket call option.

8 Conclusion

We set out to see whether a DNN could trade assets underlying a European Call option so as to replicate the value of the contingent claim. Our findings showed that the strategy devised by our neural network better hedged the basket option both in terms of mean PNL and variance PNL than the strategy informed by the Black-Scholes equation. We conclude that our data generation techniques adequately learnt the underlying distribution of the two stocks in question, as they generated paths which informed a competitive hedging strategy, and that DNN's can effectively be used to solve the short-comings of the Black-Scholes equations, in that they can more accurately replicate basket price options in incomplete, discrete-time markets.

Furthermore, suppose that X is an Itô process satisfying

$$dX_t = \mu_t dt + \sigma_t dW_t, \quad (8.1)$$

then $Y_t = f(t, X_t)$ is also an Itô process, and

$$dY_t = df(t, X_t) = \frac{\partial f}{\partial t} dt + \frac{\partial f}{\partial X_t} dX_t + \frac{1}{2} \frac{\partial^2 f}{\partial X_t^2} (\sigma_t)^2 dt.$$

In integral form, this can be written as:

$$Y_t = Y_0 + \int_0^t \left(\frac{\partial f}{\partial t} + \mu_s \frac{\partial f}{\partial X_s} + \frac{1}{2} \frac{\partial^2 f}{\partial X_s^2} (\sigma_s)^2 \right) ds + \int_0^t \frac{\partial f}{\partial X_s} \sigma_s dW_s.$$

Here, $Y_0 = f(0, X_0)$ is the initial condition.

9 Bibliography

references